

Chapter 30. Inference & Experiments

Confidence intervals, effect sizes, power and sample size, A/B test design and analysis, and the five ways a clean test goes wrong.

Draft chapter

Contents

Chapter 30. Inference & Experiments

Why this matters

In plain English

30.1 The data, and the question

30.2 Standard error and confidence intervals

30.3 Comparing two groups: the t-test

30.4 Effect size: how big, not just whether

30.5 Categorical outcomes: proportions and chi-square

30.6 More than two groups: ANOVA

30.7 Power and sample size

30.8 Designing an experiment

30.9 Riverstone's website test, end to end

30.10 Four ways to fool yourself, demonstrated

30.11 Regression for inference

30.12 Writing the result up

Common mistakes and how to spot them

In the real world: the test that "won" on a Wednesday

Tools

The project: design and analyze an experiment

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 30. Inference & Experiments

Part III — Advanced Analytics & Analytics Engineering

Chapter at a glance

You will learn to: compute a standard error and a confidence interval, by hand and in Python · compare two groups with a t-test and know when it doesn't apply · report an effect size, not only a p-value · test categorical results with a two-proportion test and chi-square · compare several groups with ANOVA without inflating your error rate · calculate the sample size a test needs before you run it · design an A/B test end to end, and write the plan down first · analyze one properly, with a sample-ratio check and guardrail metrics · recognize peeking, multiple testing, novelty effects, and outliers by watching them happen in simulations · read regression coefficients as statements about the business.

Before you start: Chapter 21 (distributions, spread, sampling) and Chapter 22 (p-values, confidence intervals, Type I and II errors) are assumed, not repeated. Chapter 29 (Python as tested code) shapes how the analysis is written; Chapter 17's pandas is enough to follow the code.

Time needed: 14–18 hours, spread over three weeks.

Tools: Python 3.12 with pandas, scipy, statsmodels, and matplotlib. Everything runs on one laptop in seconds.

Practice data: Riverstone's website data, built by `generate_riverstone_web.py` in the companion folder: 214,528 sessions, 622,090 events, and 47,286 visitors in a two-week A/B test. Fixed seed, so your numbers match the book's exactly. The one-year sales database from Chapter 13 is used for the examples about order values.

Why this matters

Riverstone's marketing agency proposes a new enquiry form. Two weeks later they report: "Enquiries are up 14%. Roll it out everywhere."

Four questions decide whether that sentence is worth anything, and all four are this chapter:

- **Is it real, or is it noise?** 14% of what? Over how many visitors? A fortnight of ordinary variation can produce bigger swings than that.
- **How big is the effect, and how sure are we?** "Up 14%" is a point estimate. The honest version is a range, and the range is often wide enough to change the decision.
- **Was the test run properly?** Were visitors split evenly? Did someone check the numbers daily and stop when they looked good? Did the agency test five things and report the one that won?
- **What do we do now?** A result nobody can act on is a cost, not a finding.

Being the person who asks these questions, calmly and with the arithmetic to back them up, is most of what separates a senior analyst from a junior one. It's also the densest interview topic in the whole book: Chapter 73's bank is full of "how would you design this test" and "here's a result, what's wrong with it".

In plain English

Inference is what you say about a whole crowd after talking to a few people in it.

Suppose you want to know the average height of everyone at a wedding, and you measure ten guests. Your ten-person average won't be exactly the crowd's average, but it won't be wildly off either. The **standard error** measures how far off it's likely to be. Measure forty guests instead of ten, and the error halves: to be twice as precise you need four times as many people, which is why sample size questions have expensive answers.

A **confidence interval** turns that into a sentence you can say out loud: "the average is 168 cm, give or take 3 cm." A **hypothesis test** answers a narrower question: "if the two sides of the room were really the same height, how surprising is the difference I measured?" Surprising enough, and you stop believing they're the same. The **p-value** is the size of that surprise, and nothing else: not the chance you're wrong, and not the size of the difference.

An experiment is how you get a clean answer instead of an argument. Rather than comparing people who happened to see the new form with people who happened not to, you flip a coin for each visitor. Once the coin decides, the two groups differ only by chance and by the form, so a difference between them points at the form.

Three things ruin this, and all three are ordinary human behavior: stopping the moment the numbers look good (**peeking**), trying twenty ideas and reporting the one that worked (**multiple testing**), and the fact that anything new attracts attention for a week (**novelty**). The rest of the chapter is arithmetic; these three are discipline.

30.1 The data, and the question

Riverstone's website, `riverstone.example`, has three tables in the companion data:

Table	One row is	Rows
<code>web_sessions</code>	one visit: device, browser, channel, region, pages viewed, and whether the visitor sent an enquiry	214,528
<code>web_events</code>	one action inside a session: a page view, a form start, a form submit	622,090
<code>ab_test_assignments</code>	one visitor in the enquiry-form test, and which version they were shown	47,286

Build them with the generator, which needs only the Python standard library:

```
# terminal, in companion/ch30
python3 generate_riverstone_web.py
# sessions 214,528  events 622,090  visitors in the test 47,286
```

The test itself, `enquiry_form_2026_02`, ran from 2 to 15 February 2026. Half the visitors saw the existing enquiry form (**control**), half saw a shorter one with fewer required fields (**variant_b**). The business question: *does the shorter form bring more enquiries?*

```
import pandas as pd

sessions = pd.read_csv("web_data/web_sessions.csv", parse_dates=["started_at"])
assignments = pd.read_csv("web_data/ab_test_assignments.csv", parse_dates=["assigned_at"])

print(sessions.shape, assignments.shape)
print(sessions[["device", "channel", "pages_viewed", "enquiry_submitted"]].head(3))
print(f"overall enquiry rate: {sessions['enquiry_submitted'].mean():.4f}")
```

```
(214528, 12) (47286, 4)
   device  channel  pages_viewed  enquiry_submitted
0  desktop  referral             1                  0
1  desktop  referral             3                  0
2  desktop  organic             2                  0
overall enquiry rate: 0.0359
```

Simplification note. Riverstone's site is a stand-in for any website with a form on it, and the data is generated, not collected. Real clickstream data is messier: bots, blocked cookies, visitors who switch devices, sessions that never end. The statistics in this chapter are exactly what you'd run on real data; the cleaning that comes first is Chapters 14 and 47.

30.2 Standard error and confidence intervals

Every sample gives a number that isn't quite the truth. The **standard error** (SE) says how much that number wobbles from sample to sample.

For an average

For a mean, the standard error is the standard deviation divided by the square root of the sample size. Compute it by hand first, on the time visitors spend on the site:

```
import numpy as np

duration = sessions["duration_seconds"]
n = len(duration)
mean = duration.mean()
sd = duration.std(ddof=1)
se = sd / np.sqrt(n)

print(f"n          = {n:,}")
print(f"mean        = {mean:.2f} seconds")
print(f"sd          = {sd:.2f}")
print(f"se          = {se:.3f}")
print(f"95% CI      = {mean - 1.96 * se:.2f} to {mean + 1.96 * se:.2f} seconds")
```

```
n          = 214,528
mean       = 96.17 seconds
sd         = 114.91
se         = 0.248
95% CI     = 95.68 to 96.65 seconds
```

Reading it: individual sessions vary enormously (a standard deviation of 115 seconds against a mean of 96), but the *average* of 214,528 of them is known to within a quarter of a second: the 95% interval is 95.68 to 96.65. That's the whole idea: the sample mean is far more stable than the thing it's averaging.

The **1.96** is the number of standard errors that contains 95% of a normal distribution. For small samples, use the t-distribution instead, which scipy will do for you:

```
from scipy import stats

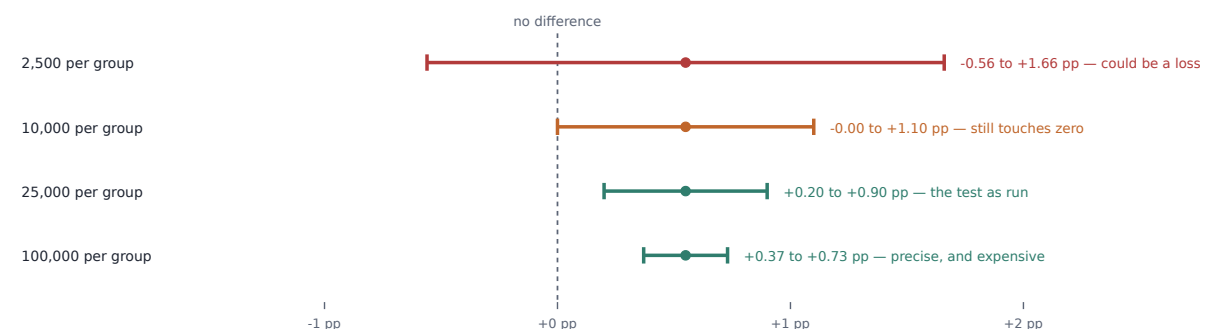
sample = sessions["duration_seconds"].sample(40, random_state=30)
low, high = stats.t.interval(0.95, df=len(sample) - 1, loc=sample.mean(),
                             scale=stats.sem(sample))
print(f"40 sessions: mean {sample.mean():.1f} s, 95% CI {low:.1f} to {high:.1f} s")
```

```
40 sessions: mean 64.0 s, 95% CI 47.8 to 80.2 s
```

Forty sessions give an interval tens of seconds wide; 214,528 give one a fraction of a second wide.

Precision costs data, and it costs it at the square root: four times the data for twice the precision.

The same lift, four sample sizes: what a confidence interval buys



Every row measured the same +0.55 percentage points. Only the sample size changed, and with it what you are entitled to say.

Figure 30.1 – The same measured lift, four sample sizes. Only the sample size changed, and with it what you may say.

For a proportion

Conversion rates are proportions, and their standard error has its own formula: the square root of $p(1 - p) / n$.

```
visitors = (sessions.groupby("visitor_id")["enquiry_submitted"].max()
            .rename("enquired").reset_index())
k = int(visitors["enquired"].sum())
n = len(visitors)
p = k / n
se = np.sqrt(p * (1 - p) / n)

print(f"{k:,} of {n:,} visitors sent an enquiry")
print(f"rate      = {p:.4f}  ({p * 100:.2f}%)" )
print(f"se       = {se:.5f}")
print(f"95% CI    = {100 * (p - 1.96 * se):.2f}% to {100 * (p + 1.96 * se):.2f}%")
```

```
7,643 of 176,000 visitors sent an enquiry
rate      = 0.0434  (4.34%)
se       = 0.00049
95% CI    = 4.25% to 4.44%
```

Watch the unit of analysis. The rate above is per *visitor*, not per *session*: a visitor who came three times and enquired once counts as one converted visitor. The test randomizes visitors, so visitors are what you count. Mixing the two is one of the most common errors in A/B analysis, and it quietly inflates your sample size, which makes everything look more significant than it is.

Watch out: a confidence interval is about the method, not the number. "95% confident" means that if you repeated this sampling many times, 95% of the intervals you built this way would contain the true value. It does *not* mean there's a 95% chance the truth is inside this particular interval; the truth is fixed, and the interval is what varies. In a meeting, say it as a range: "*between 3.9% and 4.1%*". That's what people actually need.

30.3 Comparing two groups: the t-test

"Do Wholesale customers place bigger orders than Retail ones?" This is Chapter 13's sales data, and the tool is a **two-sample t-test**: it asks how surprising the difference between two averages would be if the two groups really had the same average.

```

from scipy import stats
from sqlalchemy import create_engine

engine = create_engine("postgresql+psycopg://postgres:riverstone123@localhost/riverstone_2025")
orders = pd.read_sql("""
    SELECT o.order_id, c.segment, SUM(sl.net_revenue) AS order_value
    FROM sales_lines AS sl
    JOIN orders AS o ON o.order_id = sl.order_id
    JOIN customers AS c ON c.customer_id = sl.customer_id
    GROUP BY o.order_id, c.segment
""", engine)

retail = orders.loc[orders["segment"] == "Retail", "order_value"]
wholesale = orders.loc[orders["segment"] == "Wholesale", "order_value"]

print(f"Retail:    n = {len(retail):3d}, mean = {retail.mean():9,.0f}, sd = {retail.std():9,.0f}")
print(f"Wholesale: n = {len(wholesale):3d}, mean = {wholesale.mean():9,.0f}, sd = {wholesale.std():9,.0f}")

result = stats.ttest_ind(wholesale, retail, equal_var=False)
print(f"difference = {wholesale.mean() - retail.mean():.0f}")
print(f"t = {result.statistic:.3f}, p = {result.pvalue:.4f}, df = {result.df:.1f}")
print(f"95% CI for the difference: {result.confidence_interval().low:.0f} to {result.confidence_interval().high:.0f}")

```

```

Retail:    n = 59, mean = 25,233, sd = 15,719
Wholesale: n = 55, mean = 30,957, sd = 21,094
difference = 5,724
t = 1.634, p = 0.1055, df = 99.5
95% CI for the difference: -1,228 to 12,676

```

How it works:

- **equal_var=False** runs **Welch's t-test**, which doesn't assume the two groups have the same spread. Use it by default: it costs almost nothing when the spreads do match, and it's right when they don't.
- **t** is the difference measured in standard errors. **p** is the chance of seeing a difference at least this large if the true difference were zero.
- The **confidence interval for the difference** is the part to quote. It answers "*how much bigger?*", which is the business question; the p-value only answers "*could this be nothing?*".

What this particular result says. Wholesale orders averaged ₹30,957 against Retail's ₹25,233, a difference of ₹5,724, but $p = 0.1055$ and the interval runs from -₹1,228 to +₹12,676. The interval contains zero, so 114 orders can't tell these two segments apart: the honest answer is "*we don't know yet*", not "*Wholesale orders are 23% bigger*". That's Chapter 22's warning about small samples, now measured. Section 30.7 works out how many orders would be needed to settle it.

What the p-value does not say

People often say	What it actually means
"There's a 2% chance the result is wrong"	No. It's the chance of data this extreme <i>if there were no difference at all</i>
"p = 0.04 means the effect is real; p = 0.06 means there's nothing there"	0.05 is a convention, not a boundary in nature. Report the number and the interval

People often say	What it actually means
"p = 0.001 means the effect is big"	No. With enough data, a trivial difference gets a tiny p-value. See section 30.4
"p = 0.3 proves the two are the same"	No. It means this test couldn't tell them apart, which is also what too little data looks like

When a t-test isn't the right tool

The t-test compares means and assumes the *sample means* are roughly normally distributed. With samples of 30 or more that's usually fine even for skewed data, thanks to the central limit theorem (Chapter 21). Two situations need care:

- **Very skewed data with heavy tails**, such as Riverstone's enquiry values, where one bulk enquiry is worth twenty ordinary ones. Then the mean itself may be the wrong summary: compare medians (the Mann-Whitney U test), or analyze log values, or cap the extremes, and say which you did.
- **Data that isn't independent**: several sessions from the same visitor, or several orders from the same customer. Aggregate to the randomization unit first, as section 30.2 did.

```
enquiries = sessions.loc[sessions["enquiry_submitted"] == 1, "enquiry_value"]
print(f"n = {len(enquiries):,}")
print(f"mean   = {enquiries.mean():.0f}")
print(f"median = {enquiries.median():.0f}")
print(f"largest three: {[f'{v:,.0f}' for v in enquiries.nlargest(3)]}")
print(f"the top 1% of enquiries hold {100 * enquiries.nlargest(len(enquiries) // 100).sum() / enquiries.sum():.1f}% of the value")
```

```
n = 7,711
mean   = 41,045
median = 22,747
largest three: ['4,388,000', '2,679,214', '2,002,875']
the top 1% of enquiries hold 16.6% of the value
```

That's the shape of data where a mean comparison can swing on a single row. Section 30.10 returns to it.

30.4 Effect size: how big, not just whether

A p-value answers "*could this be nothing?*". It says nothing about *how much*, and with a large enough sample almost any difference gets a small p-value. Watch that happen:

```
rng = np.random.default_rng(30)
group_a = rng.normal(100.0, 15, 200_000)
group_b = rng.normal(100.3, 15, 200_000)    # a difference of 0.3 in 15: trivial

t = stats.ttest_ind(group_b, group_a, equal_var=False)
d = (group_b.mean() - group_a.mean()) / np.sqrt((group_a.var(ddof=1) + group_b.var(ddof=1)) / 2)
print(f"difference = {group_b.mean() - group_a.mean():.3f}")
print(f"p-value    = {t.pvalue:.6f}")
print(f"Cohen's d   = {d:.3f}")
```

```
difference = 0.298
p-value    = 0.000000
Cohen's d  = 0.020
```

A difference of three tenths of a point, on a scale where people vary by 15, is not worth a meeting. The p-value calls it significant because the sample is enormous. **Significant means "measurable", not "important".**

Effect size measures importance on a scale that doesn't grow with the sample:

Measure	For	Formula	Rough reading
Absolute difference	anything	$\text{mean}_2 - \text{mean}_1$, or $p_2 - p_1$	in the units the business uses
Relative lift	rates	$(p_2 - p_1) / p_1$	"14% more enquiries"
Cohen's d	two means	difference ÷ pooled standard deviation	0.2 small, 0.5 medium, 0.8 large
Odds ratio	proportions in regression	section 30.11	1.0 is no effect
Cramér's V	chi-square tables	section 30.5	0 to 1, higher is stronger

Riverstone's real difference, between visitors who enquire and those who don't:

```
enquired = sessions.loc[sessions["enquiry_submitted"] == 1, "duration_seconds"]
browsed = sessions.loc[sessions["enquiry_submitted"] == 0, "duration_seconds"]
d = (enquired.mean() - browsed.mean()) / np.sqrt((enquired.var(ddof=1) + browsed.var(ddof=1)) / 2)

print(f"enquired: n = {len(enquired):,}, mean = {enquired.mean():.1f} s")
print(f"browsed: n = {len(browsed):,}, mean = {browsed.mean():.1f} s")
print(f"Cohen's d = {d:.2f}")
```

```
enquired: n = 7,711, mean = 173.9 s
browsed: n = 206,817, mean = 93.3 s
Cohen's d = 0.51
```

Always report three numbers together: the effect (with its unit), the confidence interval, and the sample size. "Enquiries rose 0.55 percentage points (95% CI 0.19 to 0.91), from 3.90% to 4.45%, over 47,286 visitors" is a complete sentence. " $p < 0.01$ " is not.

Watch out: relative lift on a small base is a rhetorical device. A rise from 0.1% to 0.2% is "a 100% improvement" and also one extra enquiry per thousand visitors. Give both forms every time: the absolute change tells people what will land in their inbox, the relative one tells them how much the thing they changed mattered.

30.5 Categorical outcomes: proportions and chi-square

Most experiment metrics are proportions: converted or not, clicked or not, churned or not. Two tools cover them.

Two proportions

The difference between two rates, with its standard error, is section 30.2's arithmetic applied twice.

`statsmodels` has it built in:

```
from statsmodels.stats.proportion import proportions_ztest, confint_proportions_2indep

paid = visitors.merge(sessions[["visitor_id", "channel"]].drop_duplicates("visitor_id"),
on="visitor_id")
counts = paid.groupby("channel")["enquired"].agg(["sum", "count"])
print(counts)

k_email, n_email = counts.loc["email", "sum"], counts.loc["email", "count"]
k_paid, n_paid = counts.loc["paid", "sum"], counts.loc["paid", "count"]
stat, pvalue = proportions_ztest([k_email, k_paid], [n_email, n_paid])
low, high = confint_proportions_2indep(k_email, n_email, k_paid, n_paid, method="wald")

print(f"email {k_email / n_email:.4f} vs paid {k_paid / n_paid:.4f}")
print(f"z = {stat:.2f}, p = {pvalue:.2e}")
print(f"95% CI for the difference: {low:.4f} to {high:.4f}")
```

```
      sum  count
channel
direct  2236  42037
email    769  12485
organic  2685  66885
paid    1131  35179
referral   822  19414
email 0.0616 vs paid 0.0321
z = 14.45, p = 2.60e-47
95% CI for the difference: 0.0248 to 0.0340
```

Chi-square: several categories at once

"Does enquiry rate depend on the channel at all?" A **chi-square test of independence** compares the counts you observed with the counts you'd expect if the two columns were unrelated:

```
table = pd.crosstab(paid["channel"], paid["enquired"])
chi2, p, dof, expected = stats.chi2_contingency(table)
cramers_v = np.sqrt(chi2 / (table.to_numpy().sum() * (min(table.shape) - 1)))

print(table)
print(f"chi-square = {chi2:.1f}, dof = {dof}, p = {p:.3e}")
print(f"Cramér's V = {cramers_v:.3f}")
print("expected counts if channel made no difference:")
print(pd.DataFrame(expected, index=table.index, columns=table.columns).round(0))
```

```

enquired      0      1
channel
direct    39801  2236
email     11716   769
organic    64200  2685
paid       34048  1131
referral   18592   822
chi-square = 321.3, dof = 4, p = 2.772e-68
Cramér's V = 0.043
expected counts if channel made no difference:
enquired      0      1
channel
direct    40211.0  1826.0
email     11943.0   542.0
organic    63980.0  2905.0
paid       33651.0  1528.0
referral   18571.0   843.0

```

How it works:

- The test adds up, over every cell, how far the observed count is from the expected one, relative to the expected one. Large total, small p-value.
- **Degrees of freedom** is $(\text{rows} - 1) \times (\text{columns} - 1)$.
- A significant chi-square says "*these columns are related*", not *which* cell drives it. Compare observed against expected to see that, then test the specific pair you care about, as above.
- **Cramér's V** is the effect size: the relationship here is real and small, which is typical of channel data.
- The test needs reasonable counts in each cell (a common rule: every expected count at least 5). For rare outcomes in small samples, use Fisher's exact test instead (`stats.fisher_exact`).

Watch out: significant does not mean causal. Email visitors enquire more often than paid ones, but people on the mailing list already know Riverstone. The channel didn't cause the enquiry; the relationship with the company did. That's Chapter 22's correlation-and-causation point, and it's exactly what an experiment fixes, by deciding the groups with a coin instead of letting visitors sort themselves.

30.6 More than two groups: ANOVA

Comparing four regions with t-tests means six comparisons, and each one carries its own 5% chance of a false alarm. **Analysis of variance (ANOVA)** asks one question instead: "*is there any difference among these groups?*"

```
by_region = [g["duration_seconds"].to_numpy() for _, g in sessions.groupby("region")]
f_stat, p_value = stats.f_oneway(*by_region)
print(sessions.groupby("region")["duration_seconds"].agg(["count", "mean"]).round(1))
print(f"F = {f_stat:.3f}, p = {p_value:.4f}")

by_channel = [g["pages_viewed"].to_numpy() for _, g in sessions.groupby("channel")]
f_stat, p_value = stats.f_oneway(*by_channel)
print(sessions.groupby("channel")["pages_viewed"].agg(["count", "mean"]).round(2))
print(f"F = {f_stat:.2f}, p = {p_value:.3e}")
```

```
      count  mean
region
East    26118  95.3
North   47122  96.5
South   55502  95.6
West    85786  96.6
F = 1.413, p = 0.2367
      count  mean
channel
direct    51223  2.75
email     15264  2.77
organic   81523  2.73
paid      42822  2.73
referral  23696  2.76
F = 3.17, p = 1.298e-02
```

Reading it: the four regions behave the same: F is close to 1 and $p = 0.24$, so there's no evidence of any difference in how long visits last. The five channels do differ in pages viewed, at $p = 0.013$. ANOVA compares the variation *between* group means with the variation *within* groups, and F is the ratio: the bigger F is, the harder it is to explain the gaps between groups as noise.

ANOVA says *something* differs. To find out what, run a **post-hoc test** that corrects for the number of comparisons:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

sample = sessions.sample(20_000, random_state=30)
tukey = pairwise_tukeyhsd(sample["pages_viewed"], sample["channel"], alpha=0.05)
print(tukey.summary())
```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1 group2 meandiff p-adj  lower  upper  reject
-----
direct  email   -0.0141 0.9991 -0.1674 0.1392  False
direct  organic  -0.0024 1.0    -0.0972 0.0925  False
direct  paid      0.026 0.9676 -0.0841 0.1362  False
direct  referral  0.0403 0.9178 -0.0905 0.1712  False
email   organic  0.0118 0.9995 -0.135 0.1585  False
email   paid      0.0401 0.9571 -0.1169 0.1972  False
email   referral  0.0544 0.9105 -0.1177 0.2266  False
organic paid      0.0284 0.9398 -0.0725 0.1293  False
organic referral  0.0427 0.8788 -0.0804 0.1658  False
paid    referral  0.0143 0.9985 -0.1209 0.1495  False
-----
```

Tukey's test reports every pair with an interval and a corrected decision, where `reject = True` means the pair differs after allowing for the fact that you looked at ten pairs. **Not one pair is rejected here**, and that's the lesson, not a failure: look again at the channel means, which run from 2.73 to 2.77 pages. On 214,528 sessions that gap is measurable ($p = 0.013$); on a 20,000-row sample no individual pair stands out; and in either case it is four hundredths of a page. This is section 30.4 arriving from the other direction: before asking whether a difference is significant, ask whether the difference would change anything.

Why the correction matters, in one simulation: twenty comparisons between groups that are all identical.

```
rng = np.random.default_rng(30)
false_alarms = 0
for _ in range(20):
    a = rng.normal(100, 15, 500)
    b = rng.normal(100, 15, 500)          # exactly the same population
    if stats.ttest_ind(a, b).pvalue < 0.05:
        false_alarms += 1
print(f"comparisons: 20, 'significant' results: {false_alarms}")
print(f"chance of at least one false alarm in 20 tests at 5%: {1 - 0.95 ** 20:.1%}")
```

```
comparisons: 20, 'significant' results: 2
chance of at least one false alarm in 20 tests at 5%: 64.2%
```

Nothing differed, and yet results appeared. Run twenty tests at the 5% level and you'd expect one false alarm; the chance of at least one is 64%. That's the arithmetic behind section 30.10's multiple-testing rule.

30.7 Power and sample size

Statistical power is the chance a test detects an effect that's really there. The convention is 80%, and it depends on four things, any three of which fix the fourth:

Quantity	Meaning	Usual choice
α (alpha)	chance of a false positive you'll accept	0.05
Power ($1 - \beta$)	chance of catching a real effect	0.80
Effect size	the smallest difference worth detecting (the minimum detectable effect , MDE)	a business decision
n	sample size per group	what you're solving for

For two proportions, the formula for the sample size per group is

$$n = 2 \times (z_{\alpha/2} + z_{\beta})^2 \times p(1 - p) / d^2$$

Compute it by hand, then check it with statsmodels:

```

from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

baseline = 0.039          # Riverstone's enquiry rate before the test
mde = 0.005              # we care about half a percentage point
z_alpha, z_beta = stats.norm.ppf(1 - 0.05 / 2), stats.norm.ppf(0.80)
p_bar = baseline + mde / 2
n_by_hand = 2 * (z_alpha + z_beta) ** 2 * p_bar * (1 - p_bar) / mde ** 2

effect = proportion_effectsize(baseline + mde, baseline)
n_statsmodels = NormalIndPower().solve_power(effect_size=effect, alpha=0.05, power=0.80,
alternative="two-sided")

print(f"z_alpha = {z_alpha:.3f}, z_beta = {z_beta:.3f}")
print(f"by hand:      {n_by_hand:,.0f} visitors per group")
print(f"statsmodels: {n_statsmodels:,.0f} visitors per group")
print(f"total needed: {2 * n_statsmodels:,.0f} visitors")

```

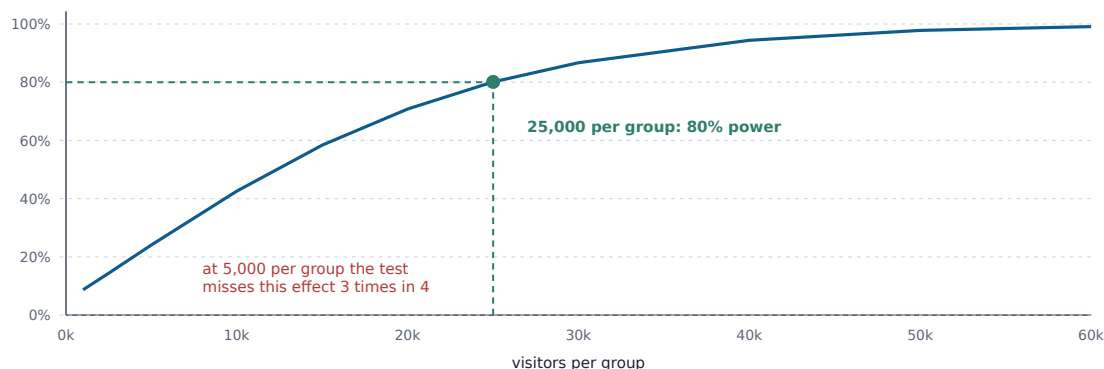
```

z_alpha = 1.960, z_beta = 0.842
by hand:      24,977 visitors per group
statsmodels: 24,955 visitors per group
total needed: 49,909 visitors

```

At about 3,400 new visitors a day, half in each group, reaching 25,000 per group takes a fortnight. Which is exactly how the fortnight in section 30.1 was chosen: **the duration came from the arithmetic, not from the calendar.**

Power: the chance of catching an effect that is really there



Detecting +0.5 percentage points on a 3.9% baseline, at the 5% level. Halving the effect you want to catch costs four times the data.

Figure 30.2 – Power against sample size. Riverstone's fortnight was chosen to land on the 80% mark.

Two more things the same machinery answers.

What effect could this test have detected? Run it backwards from the sample you actually have:

```

detectable = NormalIndPower().solve_power(nobs1=24_000, alpha=0.05, power=0.80, alternative="two-
sided")
print(f"effect size (Cohen's h) detectable with 24,000 per group: {detectable:.4f}")
print(f"that is roughly {detectable * np.sqrt(baseline * (1 - baseline)) * 100:.2f} percentage
points")

```

```

effect size (Cohen's h) detectable with 24,000 per group: 0.0256
that is roughly 0.50 percentage points

```

How many orders would settle section 30.3's question? Retail and Wholesale differed by ₹5,724 with a pooled standard deviation of about ₹18,500:

```
from statsmodels.stats.power import TTestIndPower

d = (wholesale.mean() - retail.mean()) / np.sqrt((retail.var(ddof=1) + wholesale.var(ddof=1)) / 2)
needed = TTestIndPower().solve_power(effect_size=d, alpha=0.05, power=0.80, alternative="two-sided")
print(f"observed Cohen's d = {d:.3f}")
print(f"orders needed per segment for 80% power: {needed:,.0f}")
print(f"orders available: Retail {len(retail)}, Wholesale {len(wholesale)}")
```

```
observed Cohen's d = 0.308
orders needed per segment for 80% power: 167
orders available: Retail 59, Wholesale 55
```

So the answer to "is the difference real?" is "ask again when we have three years of orders", and that's a useful thing to be able to say with a number attached.

Watch out: power calculations done afterwards are not evidence. Computing "the power we had to detect the effect we observed" (sometimes called post-hoc power) tells you nothing new: it's just the p-value in different clothes. Power belongs *before* the test, with an effect size chosen from the business, not from the data.

30.8 Designing an experiment

Everything above is arithmetic. The design is where tests are won or lost, and it belongs in a written plan agreed before a single visitor is randomized. Riverstone's plan for the enquiry-form test fits on one page:

Section	Riverstone's answer
Question	Does a shorter enquiry form bring more enquiries?
Change	variant_b : five fields instead of nine, phone number optional
Primary metric	Enquiry rate per visitor (enquiries ÷ visitors)
Guardrail metrics	Pages viewed per session; median enquiry value; enquiry-to-order rate in the CRM
Randomization unit	Visitor (not session, not page view)
Split	50/50, assigned on first session during the window
Minimum detectable effect	0.5 percentage points on a 3.9% baseline
Sample size	24,955 visitors per group, rounded to 25,000 (section 30.7)
Duration	2–15 February 2026: two full weeks, whole weeks only
Stopping rule	Run to the planned end. No decisions from interim looks

Section	Riverstone's answer
Analysis plan	Two-proportion test, 95% CI, effect size; sample-ratio check first; segments reported as exploratory only
Decision rule	Ship if the interval's lower bound is above 0 and no guardrail worsens; otherwise keep the current form

An experiment, in the order the decisions are made

Before	Question and change	the shorter enquiry form
Before	Primary metric + guardrails	enquiry rate per visitor; pages, value, orders
Before	MDE and sample size	0.5 pp → 25,000 per group
Before	Duration and stopping rule	two whole weeks, no interim decisions
During	Randomize and watch guardrails	50/50 by visitor
After	Sample-ratio check	chi-square on group sizes — failed here
After	Primary metric with CI	+0.55 pp (0.19 to 0.91), $p = 0.0026$
After	Effect size and decision	+14% relative; ship, fix the Safari tag

Everything above the line is written down before a single visitor is randomized. That is what makes the numbers below it mean anything.

Figure 30.4 — Everything above the line is decided before a visitor is randomized.

Why each choice matters:

- **One primary metric.** Choosing the winner afterwards from five metrics is multiple testing with extra steps. Everything else is a guardrail: watched to catch damage, not used to declare victory.
- **The randomization unit must match the analysis unit.** Randomize visitors, count visitors.
- **Whole weeks.** Weekday and weekend behavior differ (the data has a real weekend effect); a test that ends on a Wednesday weights midweek twice.
- **A stopping rule written in advance** is what makes the p-value mean what it says. Section 30.10 shows the damage when there isn't one.
- **Guardrails are as important as the metric.** A form that gets more enquiries by asking for less information may bring worse leads. That's why *enquiry-to-order rate* is on the list, even though it can't be measured for weeks.
- **A decision rule** turns a result into an action before anyone has a stake in the answer.

Try it. Write this table for a change you'd like to test where you work: a subject line, a page layout, a follow-up call within an hour. Most people find the *metric* and *decision rule* rows the hard ones, which is the point: they're where the disagreement lives, and it's cheaper to have it now.

30.9 Riverstone's website test, end to end

The plan is written, the fortnight has passed. Analysis follows the plan, in order, starting with the checks that can invalidate everything else.

Step 1: assemble the data at the right grain

```
test_sessions = sessions.merge(assignments, on="visitor_id")
window = (test_sessions["started_at"].dt.date >= pd.Timestamp("2026-02-02").date()) & \
        (test_sessions["started_at"].dt.date <= pd.Timestamp("2026-02-15").date())
test_sessions = test_sessions.loc[window]

per_visitor = (test_sessions.groupby(["variant", "visitor_id"])
               .agg(enquired=("enquiry_submitted", "max"),
                    sessions=("session_id", "count"),
                    pages=("pages_viewed", "sum"),
                    value=("enquiry_value", "max"))
               .reset_index())
summary = per_visitor.groupby("variant").agg(visitors=("visitor_id", "count"),
                                             enquires=("enquired", "sum"))
summary["rate"] = summary["enquires"] / summary["visitors"]
print(summary)
```

	visitors	enquires	rate
variant			
control	24036	937	0.038983
variant_b	23250	1035	0.044516

Step 2: the sample-ratio check, before anything else

A 50/50 split should produce group sizes that differ only by chance. If they don't, something is wrong with the assignment or the logging, and the outcome numbers can't be trusted either. Test it with chi-square:

```
observed = summary["visitors"].to_numpy()
chi2, p_srm = stats.chisquare(observed)
print(f"visitors: control {observed[0]:,}, variant_b {observed[1]:,}")
print(f"split: {observed[0] / observed.sum():.3%} / {observed[1] / observed.sum():.3%}")
print(f"sample-ratio mismatch test: chi-square = {chi2:.1f}, p = {p_srm:.5f}")
```

```
visitors: control 24,036, variant_b 23,250
split: 50.831% / 49.169%
sample-ratio mismatch test: chi-square = 13.1, p = 0.00030
```

This test fails. A 50.8/49.2 split of 47,286 visitors sounds harmless, but chance alone would produce a gap this large about once in 2,000 tests. That's a **sample-ratio mismatch (SRM)**, and the rule is firm: investigate before reading the result. Section 30.10 finds the cause.

Step 3: the primary metric, with an interval

```
k = summary["enquiries"].to_numpy()
n = summary["visitors"].to_numpy()
p_control, p_variant = k / n
diff = p_variant - p_control
se = np.sqrt(p_control * (1 - p_control) / n[0] + p_variant * (1 - p_variant) / n[1])
z, p_value = proportions_ztest(k[:-1], n[:-1])

print(f"control    {p_control:.4f}  ({k[0]:,} of {n[0]:,})")
print(f"variant_b  {p_variant:.4f}  ({k[1]:,} of {n[1]:,})")
print(f"absolute difference: {100 * diff:+.2f} percentage points")
print(f"relative lift:      {100 * diff / p_control:+.1f}%")
print(f"z = {z:.3f}, p = {p_value:.4f}")
print(f"95% CI (absolute):   {100 * (diff - 1.96 * se):+.2f} to {100 * (diff + 1.96 * se):+.2f} pp")
print(f"95% CI (relative):   {100 * (diff - 1.96 * se) / p_control:+.1f}% to {100 * (diff + 1.96 * se) / p_control:+.1f}%")
```

```
control    0.0390  (937 of 24,036)
variant_b  0.0445  (1,035 of 23,250)
absolute difference: +0.55 percentage points
relative lift:      +14.2%
z = 3.009, p = 0.0026
95% CI (absolute):  +0.19 to +0.91 pp
95% CI (relative):  +4.9% to +23.4%
```

The agency's "up 14%" is the middle of a range that runs from +4.9% to +23.4%. At Riverstone's traffic of about 3,400 new visitors a day, 0.55 percentage points is roughly 560 extra enquiries a month, and the interval's ends are about 190 and 920. Both ends are good news; the gap between them is the difference between hiring another person to handle enquiries and not. The range belongs in the report.

Step 4: guardrails

```
guard = per_visitor.groupby("variant").agg(pages=("pages", "mean"),
                                           sessions=("sessions", "mean"),
                                           median_value=("value", "median"))

print(guard.round(3))

enq = per_visitor.dropna(subset=["value"])
u = stats.mannwhitneyu(enq.loc[enq["variant"] == "variant_b", "value"],
                      enq.loc[enq["variant"] == "control", "value"])
print(f"enquiry value, variant_b vs control: Mann-Whitney p = {u.pvalue:.3f}")
```

```
      pages  sessions  median_value
variant
control    3.092     1.135     23066.35
variant_b  3.130     1.136     22764.70
enquiry value, variant_b vs control: Mann-Whitney p = 0.952
```

Pages viewed and sessions per visitor are unchanged, and the enquiry values are indistinguishable, so the extra enquiries show no sign of being worse. The guardrail that matters most, how many of these enquiries become orders, can't be answered for weeks, and the write-up should say so rather than implying the question is closed.

Step 5: segments, clearly labeled as exploratory

```
first = test_sessions.sort_values("started_at").drop_duplicates("visitor_id")
seg = (per_visitor.merge(first[["visitor_id", "device"]], on="visitor_id")
      .groupby(["device", "variant"])
      .agg(visitors=("visitor_id", "count"), rate=("enquired", "mean"))
      .reset_index())
print(seg.pivot(index="device", columns="variant", values=["visitors", "rate"]).round(4))
```

	visitors		rate	
variant	control	variant_b	control	variant_b
desktop	10933.0	10852.0	0.0466	0.0519
mobile	11659.0	11031.0	0.0329	0.0378
tablet	1444.0	1367.0	0.0298	0.0402

Every segment moves in the same direction, which is mildly reassuring. What you must not do is pick the biggest one and report "a 20% lift on tablet": three devices means three more chances of a false positive, and the test was never powered for a single device. Segments are for generating the next hypothesis.

30.10 Four ways to fool yourself, demonstrated

Peeking

Checking daily and stopping when p first drops below 0.05 sounds prudent. Simulate it on two groups that are identical by construction:

```
rng = np.random.default_rng(30)
rate = 0.039
peeks_that_won = 0
trials = 200

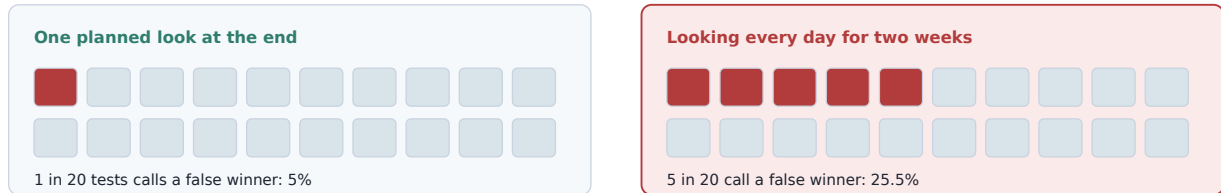
for _ in range(trials):
    a = rng.random(14_000) < rate          # two identical groups
    b = rng.random(14_000) < rate
    for day in range(1, 15):              # look once a day for two weeks
        upto = day * 1000
        _, p = proportions_ztest([a[:upto].sum(), b[:upto].sum()], [upto, upto])
        if p < 0.05:
            peeks_that_won += 1
            break

print(f"{trials} tests where nothing was different")
print(f"'significant' at some point while peeking: {peeks_that_won} ({peeks_that_won / trials:.1%})")
print("expected at a single planned look: 5%")
```

```
200 tests where nothing was different
'significant' at some point while peeking: 51 (25.5%)
expected at a single planned look: 5%
```

With no real effect at all, peeking daily turns a 5% false-positive rate into something several times larger. Run the test to its planned end, or use a method designed for looking early (sequential testing, or a Bayesian approach) and agreed in advance.

Peeking: what daily checks do to a test where nothing is different



Measured by simulation in section 30.10: 200 tests of two identical groups, 14 daily looks each, stopping at the first $p < 0.05$.

The p-value is only trustworthy for the look you promised in advance.

Figure 30.3 – Measured by simulation: peeking daily turns a 5% false-positive rate into 25%.

Multiple testing

Twenty metrics, and one or two of them will look good. The fix is to decide the primary metric first, and to correct when several must be tested:

```
from statsmodels.stats.multitest import multipletests

rng = np.random.default_rng(31)
pvalues = []
for _ in range(20):
    a = rng.normal(100, 15, 3_000)
    b = rng.normal(100, 15, 3_000)
    pvalues.append(stats.ttest_ind(a, b).pvalue)

raw = sum(p < 0.05 for p in pvalues)
bonferroni = multipletests(pvalues, alpha=0.05, method="bonferroni")[0].sum()
fdr = multipletests(pvalues, alpha=0.05, method="fdr_bh")[0].sum()
print(f"smallest p-value of 20: {min(pvalues):.4f}")
print(f"'significant' uncorrected: {raw}")
print(f"after Bonferroni: {bonferroni}")
print(f"after Benjamini-Hochberg: {fdr}")
```

```
smallest p-value of 20: 0.0403
'significant' uncorrected: 2
after Bonferroni: 0
after Benjamini-Hochberg: 0
```

Bonferroni divides the threshold by the number of tests: strict, simple, and fine for a handful.

Benjamini-Hochberg controls the share of your discoveries that are false rather than the chance of any error, which suits screening many metrics.

Novelty

Anything new draws attention. Split the test by week:

```
test_sessions = test_sessions.assign(
    week=np.where(test_sessions["started_at"].dt.date < pd.Timestamp("2026-02-09").date(), "week
1", "week 2"))
weekly = (test_sessions.groupby(["week", "variant"])["enquiry_submitted"].mean().unstack())
weekly["lift"] = 100 * (weekly["variant_b"] - weekly["control"]) / weekly["control"]
print(weekly.round(4))
```

variant	control	variant_b	lift
week			
week 1	0.0335	0.0415	23.8157
week 2	0.0354	0.0374	5.4477

Week one's lift is 24%; week two's is 5%. If the test had run for five days, the headline would have been far more exciting and far less true. Two weeks is the minimum for a website change, and a follow-up look a month later is how you find out which part of the effect survived.

Sample-ratio mismatch: finding the cause

Step 2 found an uneven split. Break the assignment down by anything that could affect tagging:

```
first = test_sessions.sort_values("started_at").drop_duplicates("visitor_id")
for column in ["device", "browser", "channel"]:
    table = pd.crosstab(first[column], first["variant"])
    table["share_control"] = table["control"] / table.sum(axis=1)
    table["srm_p"] = [stats.chisquare(row)[1] for row in table[["control",
"variant_b"]].to_numpy()]
    print(table.round(4))
    print()
```

variant	control	variant_b	share_control	srm_p
device				
desktop	10933	10852	0.5019	0.5831
mobile	11659	11031	0.5138	0.0000
tablet	1444	1367	0.5137	0.1464

variant	control	variant_b	share_control	srm_p
browser				
Chrome	13598	13604	0.4999	0.9710
Edge	2126	2127	0.4999	0.9878
Firefox	628	634	0.4976	0.8659
Other	548	593	0.4803	0.1828
Safari	7136	6292	0.5314	0.0000

variant	control	variant_b	share_control	srm_p
channel				
direct	5739	5516	0.5099	0.0356
email	1717	1664	0.5078	0.3620
organic	9139	8859	0.5078	0.0369
paid	4856	4721	0.5070	0.1677
referral	2585	2490	0.5094	0.1824

Two breakdowns flag a problem: mobile, and Safari. Safari is the cause and mobile is a symptom, because most Safari visits are on phones. Safari visitors are split 53/47 instead of 50/50, while Chrome, Edge and Firefox are clean to within a rounding error. That's a bug in the assignment script,

not chance, and it's the kind of thing that only ever turns up because someone ran the check. Riverstone's options: fix the tag and rerun, or analyze only the browsers that split correctly and treat Safari separately. What you cannot do is ignore it, because the missing visitors may not be random.

Watch out: outliers can decide a revenue test on their own. Riverstone's largest enquiry is worth more than a hundred ordinary ones (section 30.3). A revenue-per-visitor metric can be moved by which group that one enquiry lands in. Standard protections: use a rate rather than a total, cap extreme values at a percentile agreed in advance (**winsorizing**), analyze the median or a log transform, and always report how many rows the decision rests on.

30.11 Regression for inference

Chapter 19 used regression to predict. Here it's used to *explain*: the coefficients, and their intervals, are the answer.

Linear regression: what moves session length

```
import statsmodels.formula.api as smf

model_data = sessions.sample(40_000, random_state=30)
linear = smf.ols("duration_seconds ~ C(device) + C(channel) + pages_viewed", data=model_data).fit()
print(linear.summary().tables[1])
print(f"R-squared: {linear.rsquared:.3f}")
```

```
=====
              coef    std err          t      P>|t|      [0.025      0.975]
-----
Intercept          118.3293         1.571     75.343     0.000     115.251     121.408
C(device)[T.mobile]    -50.5598         1.178    -42.938     0.000    -52.868    -48.252
C(device)[T.tablet]   -24.3993         2.506     -9.736     0.000    -29.311    -19.487
C(channel)[T.email]       2.0011         2.440      0.820     0.412     -2.780      6.783
C(channel)[T.organic]    0.5978         1.493      0.400     0.689     -2.329      3.525
C(channel)[T.paid]      -2.3396         1.736     -1.348     0.178     -5.743      1.063
C(channel)[T.referral]  -2.0592         2.054     -1.003     0.316     -6.085      1.966
pages_viewed           1.5367         0.305      5.043     0.000      0.939      2.134
=====
R-squared: 0.045
```

Reading it: each coefficient is the difference associated with that category, *holding the others fixed*. The reference category (the one missing from the list) is absorbed into the intercept, so `C(device)[T.mobile]` is mobile compared with desktop. `P>|t|` is the p-value for that coefficient, and the last two columns are its 95% interval. R-squared says how much of the variation the model explains, and a low value is normal for behavioral data: people are not very predictable, and the question here is whether a difference exists, not whether we can forecast a visit.

Logistic regression: what moves the enquiry rate

For a yes/no outcome, logistic regression models the log-odds. Exponentiating a coefficient gives an **odds ratio**:

```
logit_data = per_visitor.merge(first[["visitor_id", "device", "channel"]], on="visitor_id")
logit = smf.logit("enquired ~ C(variant, Treatment('control')) + C(device) + C(channel)",
data=logit_data).fit(dis=False)
odds = pd.DataFrame({"odds_ratio": np.exp(logit.params),
                    "ci_low": np.exp(logit.conf_int()[0]),
                    "ci_high": np.exp(logit.conf_int()[1]),
                    "p": logit.pvalues})
print(odds.round(3).to_string())
```

	odds_ratio	ci_low	ci_high	p
Intercept	0.058	0.052	0.065	0.000
C(variant, Treatment('control'))[T.variant_b]	1.145	1.046	1.253	0.003
C(device)[T.mobile]	0.707	0.644	0.777	0.000
C(device)[T.tablet]	0.700	0.567	0.864	0.001
C(channel)[T.email]	1.170	0.990	1.384	0.066
C(channel)[T.organic]	0.778	0.694	0.871	0.000
C(channel)[T.paid]	0.657	0.571	0.756	0.000
C(channel)[T.referral]	0.734	0.621	0.869	0.000

Reading it: the variant's odds ratio says visitors shown the shorter form had about 15% higher odds of enquiring, with an interval that stays above 1, which agrees with section 30.9's simple two-proportion test. That agreement is the point: regression didn't rescue a weak result, it re-stated a clear one while holding device and channel constant. An odds ratio is not a relative risk; at low rates like 4% they're close, but say "odds" when you mean odds.

Watch out: control variables can't fix a broken experiment. Adding `device` to the model adjusts for imbalance you can see in the data. It does nothing about imbalance you can't, which is exactly what a sample-ratio mismatch warns you about. Randomization is what makes the comparison fair; regression only sharpens it.

30.12 Writing the result up

The analysis is done in an hour. The write-up is what people act on, and it needs four things in this order: what you found, how sure you are, what to do, and what would change your mind.

Riverstone's enquiry-form test, as Meera sends it:

Shorter enquiry form: recommend shipping, with one caveat

Over two weeks (2–15 February), 47,286 visitors were split between the current form and a shorter one. The shorter form produced **4.45% enquiries against 3.90%**, a rise of **0.55 percentage points** (95% CI 0.19 to 0.91), or **+14% relative** (95% CI +5% to +23%). At February's traffic that's roughly **560 extra enquiries a month** (the interval's ends are about 190 and 920).

Guardrails: pages viewed, sessions per visitor, and the size of the enquiries are unchanged, so the extra enquiries don't look worse on the face of it. We can't yet say how many became orders; I'll report that in six weeks.

Caveat: Safari visitors were split 53/47 rather than 50/50, a tagging bug in the assignment script. Excluding Safari does not change the direction or significance of the result, but the bug should be fixed before the next test.

Recommendation: ship the shorter form, keep the phone-number field optional, and review order conversion in six weeks. If enquiry-to-order rate drops by more than a fifth, we should revisit.

Notice what it doesn't do: it doesn't say "statistically significant" without a number, doesn't hide the bug, doesn't quote a p-value as the headline, and doesn't promise revenue it hasn't measured.

Instead of	Write
"The test was significant ($p < 0.05$)"	"4.45% against 3.90%: +0.55 points, 95% CI 0.19 to 0.91"
"Enquiries rose 14%"	"+14% relative, which is 35 enquiries a month at current traffic"
"No difference between the forms"	"No difference we could detect; a gap smaller than 0.5 points would not have shown up in this test"
"Tablet users loved it (+20%)"	"Segments moved in the same direction; the test wasn't powered for individual devices"

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reporting a p-value with no effect size	"Significant!" and nobody knows whether to act	Effect, interval, and sample size in one sentence
Treating $p = 0.06$ as "no effect"	Real effects dropped for missing a convention	Report the number and the interval; 0.05 is not a law of nature
Analyzing sessions when you randomized visitors	Inflated sample, everything looks significant	Aggregate to the randomization unit first
Skipping the sample-ratio check	A broken test read as a real result	Chi-square the group sizes before anything else
Peeking and stopping at the first good day	False positives several times the stated rate (25% in section 30.10)	Fixed end date, or a sequential method agreed in advance

Mistake	Symptom	Fix
Choosing the winning metric afterwards	Something always wins	One primary metric, written down; guardrails are not winners
Testing many segments and reporting the best	"Tablet users loved it"	Segments are exploratory; power was for the whole test
Ignoring novelty	A five-day test reads four times better than the truth	Two whole weeks minimum, then re-check later
Letting one huge value decide a revenue test	Result flips when one row moves groups	Use rates, winsorize or take medians, and say so in advance
Mean comparisons on heavy-tailed data	A t-test on data where the mean is meaningless	Median tests, logs, or capped values
Post-hoc power calculations	"We had 12% power" used as an excuse	Power before the test, from a business-chosen effect
Reading a confidence interval as "95% chance the truth is here"	Overconfident statements	"Between X and Y" is the honest phrasing
Confusing an odds ratio with a relative rate	Overstated lifts in logistic results	Say "odds"; convert if the audience needs rates
Adding controls to fix imbalance	A broken randomization treated as fixed	Controls sharpen a fair comparison; they can't create one
Running a test with no decision rule	Endless argument after the result arrives	Agree "ship if..." before the data exists
Rounding away the uncertainty in a headline	" +14%" repeated until it becomes a fact	Carry the interval into the summary line

In the real world: the test that "won" on a Wednesday

The agency's email lands on Wednesday 4 February, two days into the fortnight: *"Early results are excellent: the new form is up 36%, significant at $p = 0.003$. We suggest rolling it out now and saving the remaining spend."*

Meera opens the plan the three of them signed on 30 January. It says: two full weeks, one look, decision rule agreed. She replies with four sentences and a request to wait.

Two weeks later the result is +14%, not +36%, with an interval of +5% to +23%. Three things had made Wednesday's number what it was:

- **Two days of data.** 7,592 visitors, about a sixth of the planned sample. At that size the test had a fair chance of detecting only differences of about 1.5 percentage points or more, so anything it did detect was bound to look enormous.
- **Novelty.** The first days of any change draw disproportionate attention; week one's lift was 24% against week two's 5%.
- **Peeking.** The agency had checked daily since the test started. Section 30.10's simulation puts the false-positive rate for that habit at about 25%, not 5%.

The ending is the good kind: the form really is better, and Riverstone shipped it. But the decision rested on a number two and a half times too large, from a test that had a one-in-four chance of "winning" on noise alone, and it nearly cost the company the rest of the fortnight's data and the Safari bug that the final check uncovered.

What Meera tells Anita: *"The form works, and we're shipping it. The early number was two and a half times too big, which is what early numbers usually are. The agreement to wait two weeks is what let us report a range we can stand behind, and it's also why we found the tagging bug. I'd like the same one-page plan for every test we run with them."*

Notice that the discipline was not statistical skill. It was a page written before the test, and one person willing to say "wait" to a confident email.

Tools

Versions used for this chapter, checked in September 2026:

- **Python 3.12.3, pandas 3.0.2, numpy 2.4.4, scipy 1.17.1, statsmodels 0.15.0.** Install them with `uv add pandas scipy statsmodels` (Chapter 29).
- **scipy.stats** for t-tests, chi-square, ANOVA, and distributions. **statsmodels** for proportion tests, power and sample size, multiple-testing corrections, Tukey, and regression with readable summaries.
- **PostgreSQL** to hold the website data if you'd rather query it in SQL:
`web_data/load_postgresql.sql` builds `riverstone_web`.
- **An experiment platform** (Optimizely, VWO, GrowthBook, Statsig, or a warehouse-native setup) is what companies use to assign visitors and compute results. They automate the arithmetic here, including sample-ratio checks; they do not decide your metric, your MDE, or your stopping rule.
- **Companion files** in `ch30/`: `generate_riverstone_web.py` (the dataset, seed 30) and `ch30_check.py` (checks the chapter's numbers).

Tool note: Bayesian A/B testing. Some teams report "a 93% chance the variant is better, and a 7% chance it's worse by more than 0.2 points" instead of a p-value. That's the Bayesian framing: it starts from a prior belief, updates it with the data, and produces a probability distribution for the effect. It's popular because its sentences match how people think, and because stopping early is less damaging when the method expects it. It needs a prior, and a prior is a judgment that has to be defensible. The design work in section 30.8 is identical either way: metric, randomization, guardrails, duration, decision rule.

The project: design and analyze an experiment

Goal: a complete, honest experiment write-up: a plan agreed in advance, an analysis that follows it, and a recommendation someone can act on.

Option A: your own data. Any A/B test, pilot, or before-and-after change at work. If you have no experiment, design one on paper and analyze a past change as best you can, saying clearly what the design couldn't rule out.

Option B: Riverstone. The companion data has a second, unanalyzed change: the `/bulk-enquiry` landing page. Design a test for it, use the existing data to estimate the baseline and the sample size, and then analyze the enquiry-form test yourself from scratch before comparing with section 30.9.

Steps:

1. **Write the one-page plan** from section 30.8: question, change, primary metric, guardrails, randomization unit, MDE, sample size, duration, stopping rule, analysis plan, decision rule.
2. **Justify the MDE in money.** How many extra enquiries, orders, or rupees would make the change worth shipping? That number, not a convention, sets the sample size.
3. **Check the split** before looking at the outcome.
4. **Compute the primary metric** with its confidence interval and effect size, both absolute and relative.
5. **Check the guardrails**, and name the one you can't measure yet.
6. **Look for novelty** by comparing the first and second halves.
7. **Write the summary** in the shape of section 30.12: finding, uncertainty, recommendation, and what would change your mind.
8. **Have someone read it** who wasn't involved, and ask them what they would do next. If they can't tell, rewrite it.

Deliverables: the plan, the analysis notebook or script, and a summary of at most 300 words.

Stretch goals:

- Redo the analysis with a Bayesian two-proportion model and compare the sentences you'd say.
- Estimate how long the test would need to run to detect a 0.2 percentage point effect, and decide whether that test is worth running at all.
- Analyze the test excluding Safari visitors, and report whether the conclusion changes.
- Turn the analysis into a tested package (Chapter 29) with the reconciliation numbers as tests.

You've got it when...

- ☐ I can compute a standard error and a confidence interval for a mean and for a proportion, by hand.
- ☐ I report an effect size and an interval, never a p-value alone.

- ☐ I can say what a p-value does and doesn't mean, in one sentence, to a non-technical colleague.
 - ☐ I choose the right test for the data: Welch's t-test, two proportions, chi-square, or ANOVA with a post-hoc correction.
 - ☐ I compute the sample size before a test, from an effect size the business chose.
 - ☐ I write the plan first: metric, guardrails, randomization unit, duration, stopping rule, decision rule.
 - ☐ I check the sample ratio before I look at the outcome.
 - ☐ I can explain peeking, multiple testing, and novelty, with numbers.
 - ☐ I read regression coefficients and odds ratios as statements about the business, with their intervals.
 - ☐ My write-ups say what would change my mind.
-

Recap

- **Standard error** measures how much a sample statistic wobbles; it shrinks with the square root of n . A **confidence interval** turns it into a range you can say out loud.
 - A **p-value** is the chance of data this extreme if there were no effect. It is not the size of the effect, not the chance of being wrong, and not proof of no difference.
 - **Effect size** (absolute difference, relative lift, Cohen's d , odds ratio, Cramér's V) is what makes a result actionable. Report effect, interval, and n together.
 - **Welch's t-test** compares two means; **two-proportion tests** and **chi-square** compare rates and categories; **ANOVA** with a **post-hoc test** compares several groups without inflating the error rate.
 - **Power** is the chance of catching a real effect. Fix alpha, power, and the **minimum detectable effect**, and the **sample size** follows. Do it before the test, never after.
 - An **experiment** is designed on one page: question, change, primary metric, guardrails, randomization unit, split, MDE, sample size, duration, stopping rule, analysis plan, decision rule.
 - Analyze in order: **sample-ratio check**, primary metric with interval, effect size, guardrails, then segments as exploratory.
 - **Peeking** (25% false positives in the chapter's simulation), **multiple testing**, **novelty**, **SRM**, and **outliers** are the five ways a clean test goes wrong.
 - **Regression** answers "how much, holding other things constant": coefficients for means, **odds ratios** for yes/no outcomes, always with intervals.
 - A result is finished when someone can act on it and knows what would change your mind.
-

Practice exercises

Work in `companion/ch30`, with the data built by `generate_riverstone_web.py`. Predict each answer before running it.

Warm-up

1. Compute the 95% confidence interval for the mean number of pages viewed per session, by hand and with scipy. Do they agree?
2. What is the enquiry rate for mobile visitors, with its interval? Is it different from desktop's?
3. A colleague says " $p = 0.2$, so the two groups are the same". Rewrite the sentence honestly.
4. Take 200 random sessions and compute the mean duration's interval. Take 2,000. Take 20,000. What happens to the width, and by how much?

Core

5. Test whether the enquiry rate differs between the `email` and `organic` channels. Report z , p , the difference, and its interval.
6. Run a chi-square test of device against enquiry, and compute Cramér's V . What does it tell you that the chi-square alone doesn't?
7. Riverstone wants to detect a 0.3 percentage point improvement on a 3.9% baseline. How many visitors per group, at 80% power? How long would that take at 3,400 new visitors a day?
8. Re-analyze the A/B test excluding Safari visitors. Does the conclusion change? Report both results side by side.
9. Compute the test's result per week, and say which week you'd quote to the board and why.
10. Compare enquiry *values* between the two variants with a t-test and with Mann-Whitney. Explain why they disagree, or why they agree.

Stretch

11. Simulate the peeking experiment with weekly looks instead of daily. How much of the damage does that remove?
12. Simulate 500 A/A tests (both groups identical) at the planned sample size, and plot the p-values. What shape should they have, and do they?
13. Fit a logistic regression of enquiry on variant, device, channel, and region. Which coefficients keep intervals away from 1, and what would you tell the marketing team?
14. The agency proposes testing four form variants at once. Write the design, including how you'd control the error rate, and estimate the sample size per group.

Think about it (no code needed)

15. A test shows +0.4 percentage points, 95% CI -0.1 to +0.9. What do you recommend, and what do you need to say about it?
16. Why is randomizing by visitor rather than session important for this particular test? Give one way randomizing by session could mislead.
17. A stakeholder asks for the test to keep running "until it's significant". Explain the problem in two sentences, without using the word p-value.
18. Riverstone's enquiry-to-order rate can only be measured six weeks after the test ends. How would you handle the decision in the meantime?

Key terms

inference · population · sample · standard error · confidence interval · t-distribution · proportion · unit of analysis · hypothesis test · null hypothesis · p-value · Type I error · Type II error · Welch's t-test · Mann-Whitney U · effect size · Cohen's d · relative lift · absolute difference · odds ratio · Cramér's V · two-proportion test · chi-square test of independence · expected counts · Fisher's exact test · ANOVA · F-statistic · post-hoc test · Tukey HSD · familywise error rate · Bonferroni correction · Benjamini-Hochberg · false discovery rate · statistical power · alpha · beta · minimum detectable effect · sample size calculation · A/B test · control · variant · randomization unit · guardrail metric · primary metric · stopping rule · decision rule · sample-ratio mismatch · peeking · sequential testing · novelty effect · winsorizing · linear regression for inference · logistic regression · log-odds · Bayesian A/B testing · prior

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 31, Causal Inference Without Experiments**, answers the same questions when you can't randomize: difference-in-differences, matching, and regression discontinuity.
 - **Chapter 22, Statistics Without Fooling Yourself**, is the intuition this chapter computes; reread its correlation-and-causation section after this one.
 - **Chapter 42, Digital & Web Analytics**, uses the same website data for funnels, attribution, and cohorts.
 - **Chapter 29, Python as Software, Not Scripts**, is how an analysis like this becomes a repeatable, tested pipeline instead of a notebook nobody can rerun.
 - **Chapter 55, Machine Learning in Production**, runs experiments on models, where the treatment is a model version and the guardrails are latency and fairness.
 - **Chapter 73, Statistics, Probability & Experimentation Bank**, has about 70 interview questions, including three A/B test debugging cases built on exactly the faults in section 30.10.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```
pages = sessions["pages_viewed"]
se = pages.std(ddof=1) / np.sqrt(len(pages))
by_hand = (pages.mean() - 1.96 * se, pages.mean() + 1.96 * se)
by_scipy = stats.t.interval(0.95, df=len(pages) - 1, loc=pages.mean(), scale=stats.sem(pages))
print(f"mean {pages.mean():.4f}")
print(f"by hand: {by_hand[0]:.4f} to {by_hand[1]:.4f}")
print(f"by scipy: {by_scipy[0]:.4f} to {by_scipy[1]:.4f}")
```

```
mean 2.7413
by hand: 2.7334 to 2.7492
by scipy: 2.7334 to 2.7492
```

They agree to four decimals: with 214,528 rows the t-distribution is indistinguishable from the normal one. The difference only matters below about 30 observations.

2.

```
device = (visitors.merge(sessions.drop_duplicates("visitor_id")[["visitor_id", "device"]],
on="visitor_id")
            .groupby("device")["enquired"].agg(["sum", "count"]))
device["rate"] = device["sum"] / device["count"]
device["se"] = np.sqrt(device["rate"] * (1 - device["rate"]) / device["count"])
device["low"] = device["rate"] - 1.96 * device["se"]
device["high"] = device["rate"] + 1.96 * device["se"]
print(device.round(4).to_string())

stat, p = proportions_ztest([device.loc["mobile", "sum"], device.loc["desktop", "sum"]],
                           [device.loc["mobile", "count"], device.loc["desktop", "count"]])
print(f"mobile vs desktop: z = {stat:.2f}, p = {p:.3e}")
```

	sum	count	rate	se	low	high
device						
desktop	4183	81062	0.0516	0.0008	0.0501	0.0531
mobile	3049	84473	0.0361	0.0006	0.0348	0.0374
tablet	411	10465	0.0393	0.0019	0.0356	0.0430
mobile vs desktop: z = -15.43, p = 1.011e-53						

Mobile visitors enquire much less often than desktop ones, and the intervals don't overlap. That's a real difference and a large one in business terms, though it says nothing about *why*: people on phones may be browsing rather than buying.

3. "This test couldn't detect a difference between the groups. With the sample we had, anything smaller than about X would have been invisible, so 'no difference' and 'a difference too small for us to see' look the same here." Add the interval: if it runs from -2% to +3%, say so, because that rules out large effects even though it rules out nothing small.

4.

```
for n in (200, 2_000, 20_000):
    sample = sessions["duration_seconds"].sample(n, random_state=30)
    low, high = stats.t.interval(0.95, df=n - 1, loc=sample.mean(), scale=stats.sem(sample))
    print(f"n = {n:>6,}: mean {sample.mean():6.1f} s, CI {low:6.1f} to {high:6.1f}, width {high - low:5.1f} s")
```

n =	200:	mean	82.5 s, CI	69.6 to	95.4, width	25.8 s
n =	2,000:	mean	96.7 s, CI	91.0 to	102.4, width	11.4 s
n =	20,000:	mean	97.2 s, CI	95.6 to	98.9, width	3.3 s

Each tenfold increase in data shrinks the width by roughly $\sqrt{10} \approx 3.2$ times: 25.8 seconds, then 11.4, then 3.3. The first step falls a little short of the rule because a 200-row sample is itself a lottery, which is the whole reason small samples are treated with suspicion.

5.

```
k_email, n_email = counts.loc["email", "sum"], counts.loc["email", "count"]
k_org, n_org = counts.loc["organic", "sum"], counts.loc["organic", "count"]
stat, p = proportions_ztest([k_email, k_org], [n_email, n_org])
low, high = confint_proportions_2indep(k_email, n_email, k_org, n_org, method="wald")
print(f"email {k_email / n_email:.4f} organic {k_org / n_org:.4f}")
print(f"difference {100 * (k_email / n_email - k_org / n_org):+.2f} pp, z = {stat:.2f}, p = {p:.2e}")
print(f"95% CI: {100 * low:+.2f} to {100 * high:+.2f} pp")
```

```
email 0.0616 organic 0.0401
difference +2.15 pp, z = 10.78, p = 4.08e-27
95% CI: +1.70 to +2.59 pp
```

Email converts about 2.2 percentage points better than organic search, and the interval is nowhere near zero. As in section 30.5, the channel didn't cause this: the mailing list is made of people who already know Riverstone.

6.

```
device_table = pd.crosstab(
    visitors.merge(sessions.drop_duplicates("visitor_id"))["visitor_id", "device"],
    on="visitor_id")["device"],
    visitors.merge(sessions.drop_duplicates("visitor_id"))["visitor_id", "device"],
    on="visitor_id")["enquired"]
chi2, p, dof, expected = stats.chi2_contingency(device_table)
v = np.sqrt(chi2 / (device_table.to_numpy().sum() * (min(device_table.shape) - 1)))
print(device_table)
print(f"chi-square = {chi2:.1f}, dof = {dof}, p = {p:.3e}, Cramér's V = {v:.3f}")
```

```
enquired      0      1
device
desktop    76879  4183
mobile     81424  3049
tablet     10054   411
chi-square = 244.1, dof = 2, p = 9.815e-54, Cramér's V = 0.037
```

The chi-square says device and enquiring are related, and with 176,000 visitors it would say that for almost any real difference. Cramér's V says the relationship is weak in absolute terms. Both are true: the effect is small per visitor and large in total, because the site gets a lot of visitors.

7.

```
needed = NormalIndPower().solve_power(effect_size=proportion_effectsize(0.039 + 0.003, 0.039),
                                       alpha=0.05, power=0.80, alternative="two-sided")
print(f"visitors per group: {needed:,.0f}")
print(f"total: {2 * needed:,.0f}")
print(f"days at 3,400 new visitors a day: {2 * needed / 3400:,.0f}")
```

```
visitors per group: 67,756
total: 135,512
days at 3,400 new visitors a day: 40
```

Detecting 0.3 points instead of 0.5 takes nearly three times the visitors and about six weeks. That's the trade-off to put in front of the business: a longer test, or accepting that small improvements stay invisible.

8.

```
no_safari = test_sessions.loc[test_sessions["browser"] != "Safari"]
per_visitor_ns = (no_safari.groupby(["variant", "visitor_id"])
["enquiry_submitted"].max().reset_index())
g = per_visitor_ns.groupby("variant")["enquiry_submitted"].agg(["sum", "count"])
z, p = proportions_ztest(g["sum"].to_numpy()[::-1], g["count"].to_numpy()[::-1])
rate = g["sum"] / g["count"]
print(g.assign(rate=rate.round(4)).to_string())
print(f"difference {100 * (rate['variant_b'] - rate['control']):+.2f} pp, z = {z:.2f}, p = {p:.4f}")
print(f"split: {100 * g.loc['control', 'count'] / g['count'].sum():.2f}% control")
```

	sum	count	rate
variant			
control	679	16900	0.0402
variant_b	780	16958	0.0460
difference	+0.58	pp, z = 2.64,	p = 0.0084
split:	49.91%	control	

Without Safari the split is even and the conclusion holds: same direction, similar size, still significant. That's the sentence the write-up needs, because "we found a bug" and "the result survives it" are two different facts and the reader deserves both.

9.

```
weekly = test_sessions.groupby(["week", "variant"])["enquiry_submitted"].mean().unstack()
weekly["lift_pct"] = 100 * (weekly["variant_b"] - weekly["control"]) / weekly["control"]
print(weekly.round(4).to_string())
```

variant	control	variant_b	lift_pct
week			
week 1	0.0335	0.0415	23.8157
week 2	0.0354	0.0374	5.4477

Neither week alone: quote the fortnight, +14%, and mention that week one ran hotter because the form was new. A board deck showing week one's 24% would be repeating the agency's mistake with better arithmetic.

10.

```
vals = per_visitor.dropna(subset=["value"])
control_v = vals.loc[vals["variant"] == "control", "value"]
variant_v = vals.loc[vals["variant"] == "variant_b", "value"]
t = stats.ttest_ind(variant_v, control_v, equal_var=False)
u = stats.mannwhitneyu(variant_v, control_v)
print(f"means: control {control_v.mean():.0f} variant_b {variant_v.mean():.0f}")
print(f"medians: control {control_v.median():.0f} variant_b {variant_v.median():.0f}")
print(f"t-test p = {t.pvalue:.3f}; Mann-Whitney p = {u.pvalue:.3f}")
```

```
means:    control 42,204  variant_b 40,772
medians:  control 23,066  variant_b 22,765
t-test p = 0.700;  Mann-Whitney p = 0.952
```

Both tests agree that there's no detectable difference in enquiry value, which is the guardrail passing. They can disagree when a handful of enormous values pull one mean around; the median test ignores their size and only counts which side they fall on. When they disagree, look at the largest few rows before believing either.

11.

```
rng = np.random.default_rng(30)
wins = 0
for _ in range(200):
    a = rng.random(14_000) < 0.039
    b = rng.random(14_000) < 0.039
    for look in (7_000, 14_000): # one look mid-way, one at the end
        _, p = proportions_ztest([a[:look].sum(), b[:look].sum()], [look, look])
        if p < 0.05:
            wins += 1
            break
print(f"false 'winners' with two looks: {wins} of 200 ({wins / 200:.1%})")
```

```
false 'winners' with two looks: 15 of 200 (7.5%)
```

Two looks instead of fourteen brings the false-positive rate most of the way back towards 5%. The damage grows with the number of looks, which is why "just check on Friday" is so much safer than "check every morning", and why sequential methods exist for teams that must be able to stop early.

12.

```
rng = np.random.default_rng(30)
pvalues = []
for _ in range(500):
    a = rng.random(24_000) < 0.039
    b = rng.random(24_000) < 0.039
    pvalues.append(proportions_ztest([a.sum(), b.sum()], [24_000, 24_000])[1])
pvalues = np.array(pvalues)
print(f"below 0.05: {(pvalues < 0.05).mean():.1%}    below 0.10: {(pvalues < 0.10).mean():.1%}")
print(f"below 0.50: {(pvalues < 0.50).mean():.1%}    below 0.90: {(pvalues < 0.90).mean():.1%}")
```

```
below 0.05: 6.0%    below 0.10: 11.2%
below 0.50: 49.0%    below 0.90: 90.6%
```

When nothing is different, p-values are spread evenly between 0 and 1: about 5% below 0.05, half below 0.5. A histogram of them should look flat. That's what "a 5% false-positive rate" means, and running A/A tests like this is how teams check that their experiment platform is honest before trusting it with real ones.

13.

```

full = per_visitor.merge(first[["visitor_id", "device", "channel", "region"]], on="visitor_id")
model = smf.logit("enquired ~ C(variant, Treatment('control')) + C(device) + C(channel) +
C(region)",
                data=full).fit(dispen=False)
out = pd.DataFrame({"odds_ratio": np.exp(model.params), "low": np.exp(model.conf_int()[0]),
                    "high": np.exp(model.conf_int()[1]), "p": model.pvalues})
print(out.round(3).to_string())

```

	odds_ratio	low	high	p
Intercept	0.057	0.049	0.067	0.000
C(variant, Treatment('control'))[T.variant_b]	1.145	1.046	1.253	0.003
C(device)[T.mobile]	0.707	0.644	0.777	0.000
C(device)[T.tablet]	0.700	0.567	0.864	0.001
C(channel)[T.email]	1.170	0.990	1.384	0.066
C(channel)[T.organic]	0.778	0.694	0.871	0.000
C(channel)[T.paid]	0.657	0.571	0.756	0.000
C(channel)[T.referral]	0.734	0.621	0.869	0.000
C(region)[T.North]	1.050	0.894	1.234	0.552
C(region)[T.South]	1.036	0.885	1.212	0.659
C(region)[T.West]	1.006	0.867	1.167	0.936

The variant, device, and channel coefficients keep their intervals away from 1; region's don't, which matches section 30.6's ANOVA finding that regions behave alike. For marketing: mobile visitors convert at around 0.7 times the odds of desktop ones, and paid traffic at about two thirds the odds of direct traffic (the reference category), so the mobile form and the paid landing pages are where the next tests belong.

14. Four variants means three comparisons against control, so the error rate needs handling and the sample grows:

- Test all four against one control with a **Bonferroni-corrected alpha** of $0.05 / 3 = 0.0167$ for the comparisons, or use Dunnett's test, designed for many-versus-one.
- The stricter alpha costs sample: at 0.0167 and 80% power for a 0.5-point effect, each group needs about 33,000 visitors rather than 25,000, and there are four groups instead of two.
- Four groups of 33,000 is 133,000 visitors; at 3,400 new visitors a day that's about **six weeks**, long enough that the site, the season, and the traffic mix will all have moved.
- The honest recommendation is usually: test the one or two variants with a real hypothesis behind them, not four because the tool allows it. If all four must run, agree in advance which comparison is primary.

15. The point estimate is positive, the interval crosses zero, so the test didn't settle it. Say: "The best estimate is +0.4 points, but the range runs from -0.1 to +0.9, so this test can't tell us whether the change helps or does nothing. It does rule out anything larger than about a point." Then choose deliberately: ship it anyway if it's cheap and the guardrails are fine (a coin-flip with a slight tilt in your favor), run longer if a decision must be right, or drop it if the upside at +0.9 still wouldn't pay for the work. What you must not do is call it a win.

16. Visitors come back: 12% of them have more than one session in the fortnight. If sessions were randomized, the same person could see both forms, which contaminates the comparison, and their sessions wouldn't be independent, which breaks the arithmetic behind the test. A specific way it

misleads: people who enquire tend to browse more in that session, so the winning form would accumulate extra sessions, and session-level rates would drift apart for a reason that has nothing to do with the form.

17. *"If we keep looking until the numbers look good, we'll always find a moment when they do, even when the two forms are identical. I ran that experiment on data where nothing was different, and a quarter of the tests produced a 'winner' at some point."* Then offer the alternative: a fixed end date, or a method designed for stopping early, agreed now rather than mid-test.

18. Ship on the evidence you have, and put the six-week check in the plan as a commitment with a threshold attached: *"We'll review enquiry-to-order rate on 31 March; if it has dropped by more than a fifth, we revert."* Two supporting moves: track a faster proxy in the meantime (whether the sales team can reach the enquirer, which is known within days), and keep the old form available so reverting is a configuration change rather than a project.